

Conditional noexcept specifiers in compound requirements

Document #: P3822R0 [[Latest](#)] [[Status](#)]
Date: 2025-09-01
Project: Programming Language C++
Audience: EWGi
Reply-to: Viacheslav Luchkin
<someone12469@gmail.com>
Gašper Ažman
<gasper.azman@gmail.com>

Contents

- [1 Introduction](#)
- [2 Motivation](#)
- [3 Proposal](#)
- [4 Proposed wording](#)

§ 1 Introduction

This paper extends compound requirements to allow noexcept specifiers to be applied conditionally. The proposed syntax for this is `requires { { expression } noexcept(condition) -> return-type-constraint; }`.

§ 2 Motivation

Requires-expressions can be used to assert that an expression is non-throwing, but do not provide a way to do so conditionally, which is sometimes needed in generic programming. Achieving this now usually requires code duplication.

The lack of support for this syntax is also inconsistent with function declarations, which have had conditional noexcept specifiers since C++11.

Motivational example:

Before	After
<pre> template<typename F, bool noexc> concept invocable = noexc ? requires(F f) { { f() } noexcept; } : requires(F f) { f(); }; template<bool noexc> struct callable_ref { callable_ref(invocable<noexc> auto&& fn); [...] }; </pre>	<pre> template<typename F, bool noexc> concept invocable = requires(F f) { { f() } noexcept(noexc); }; template<bool noexc> struct callable_ref { callable_ref(invocable<noexc> auto&& fn); [...] }; </pre>

§ 3 Proposal

Redefine the syntax for compound requirements as follows:

compound-requirement:

```
{ expression } noexceptopt noexcept-specifieropt return-type-requirementopt ;
```

noexcept-specifier:

```
noexcept(condition)
noexcept
```

return-type-requirement:

```
-> type-constraint
```

Where the *condition*, if supplied, shall be a contextually converted constant expression of type `bool`. The *noexcept-specifier* `noexcept` without a *condition* is equivalent to `noexcept(true)`.

If *condition* evaluates to `true` and *expression* is a potentially-throwing expression, the requirement is not satisfied. If *condition* evaluates to `false` or the *noexcept-specifier* is absent, *expression* may be potentially-throwing.

If the conversion of *condition* to `bool` fails, the requirement is not satisfied. This is not a hard error, because requirements are often used in overload resolution and the expected behavior for an unexpected input type is that other overloads are still considered.

§ 4 Proposed wording

Recall that the grammar for *noexcept-specifier* is defined in section 14.5 [\[except.spec\]](#) as follows:

noexcept-specifier:

```
noexcept(constant-expression)
```

noexcept

Alter section 7.5.8.4 [expr.prim.req.compound] as follows:

compound-requirement:

`{ expression } noexceptopt noexcept-specifieropt return-type-requirementopt ;`

return-type-requirement:

`-> type-constraint`

- ¹ A *compound-requirement* asserts properties of the *expression E*. The *expression* is an unevaluated operand. Substitution of template arguments (if any) and verification of semantic properties proceed in the following order:
 - (1.1) — Substitution of template arguments (if any) into the *expression* is performed.
 - (1.2) — If the `noexcept` specifier is present, *E* shall not be a potentially-throwing expression ([except.spec])
 - (1.2) — If the *noexcept-specifier* ([except.spec]) is present, then:
 - (1.2.1) — Substitution of template arguments (if any) into its associated *constant-expression* is performed. The *noexcept-specifier* `noexcept` without a *constant-expression* is equivalent to `noexcept(true)`.
 - (1.2.2) — The *constant-expression* shall be a contextually converted constant expression of type `bool` ([expr.const]). If the *constant-expression* is true, *E* shall not be a potentially-throwing expression ([except.spec]).

Bump feature-test macro `__cpp_concepts` in section 15.12 [cpp.predefined]:

```
__cpp_char8_t 202207L
- __cpp_concepts 202002L
+ __cpp_concepts 20XXXXL
__cpp_conditional_explicit 201806L
```